

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³
SOICT 2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³
SOICT 2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

Good morning. I will show what a second edge board actually buys you when you split an ML intrusion detector across two Jetson Orin Nano Super devices. Seven minutes, four results.

I. PROBLEM AND CONTRIBUTIONS

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson
Orin Nano Super Edge Cluster using Kafka and PySpark
└ I. PROBLEM AND CONTRIBUTIONS

I. PROBLEM AND
CONTRIBUTIONS

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

- 1. Everything is measured on a **single device**.
- 2. Throughput, latency and **energy** are never reported together.
- 3. Offline scores come from **leaky protocols**.

Our question

How should inference be *placed* across a constrained two-node edge cluster?

Our answer

Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

I. PROBLEM AND CONTRIBUTIONS

└ Edge intrusion detection: three open gaps

Three gaps: single-device evaluation, energy never reported next to latency, and offline numbers taken from leaky pipelines. Which model to deploy is the other half of the question. We attack all three with one system.

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

- 1. Everything is measured on a **single device**.
- 2. Throughput, latency and **energy** are never reported together.
- 3. Offline scores come from **leaky protocols**.

Our question

How should inference be placed across a constrained two-node edge cluster?

Our answer

Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2 joined by a Kafka topic.

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

I. PROBLEM AND CONTRIBUTIONS

Contributions

Four contributions. The fourth matters: the model we deploy is not a fresh model, it is the one that survived a strict offline audit.

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

2026-09-30

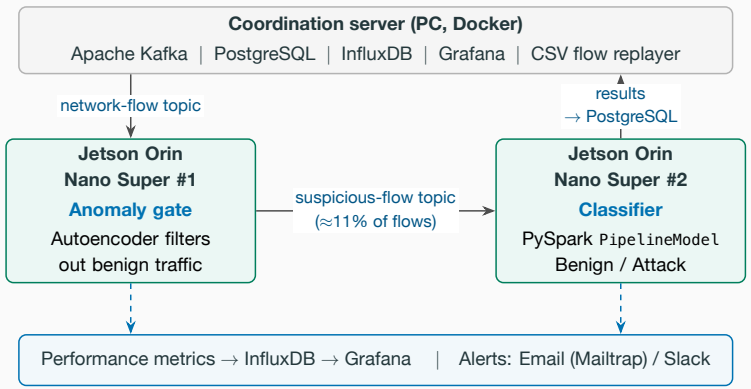
A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

└─ II. SYSTEM AND EVALUATION SETUP

II. SYSTEM AND EVALUATION
SETUP

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture



The gate scores all 77 features; its threshold is fixed *offline* on benign validation data, never on the test set. Batches of 20, or after 1 s.

CICIDS2017: Sharafaldin et al., ICISSP 2018 · Kafka: Kreps et al., NetDB 2011 · Spark: Zaharia et al., HotCloud 2010.

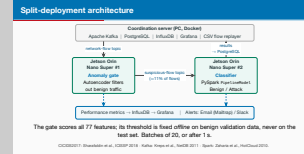
2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture

Host runs the infrastructure. Board one gates, board two classifies. Only suspicious flows cross the Kafka topic between them. Everything is instrumented into Grafana.



Deployment modes and what we measure

- Single node** full pipeline on one board, gate off or on.
- A: Pipeline split** gate on #1, classifier on #2 (our design).
- B: Horizontal** both boards run the full role (gate off).
- C: Spark cluster** host as master, both boards as workers.

Every flow is followed from send to verdict (queuing included). **Sustained capacity** the highest rate at which every flow gets its verdict within 10 s of the load.

Setup

2× Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN
10,000 test-split flows, 15% attacks
Rate sweep 1–40 flows/s, 2 runs each
Energy via tegrastats

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

II. SYSTEM AND EVALUATION SETUP

Latency is measured from the moment a flow is sent to the moment its verdict is written, so time spent waiting in a queue counts. A fresh Kafka consumer group per run, so no run inherits another’s backlog.

Deployment modes and what we measure

Single node full pipeline on one board, gate off or on.

A: Pipeline split gate on #1, classifier on #2 (our design).

B: Horizontal both boards run the full role (gate off).

C: Spark cluster host as master, both boards as workers.

Setup

2 × Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)
PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN
10,000 test-split flows, 15% attacks
Rate sweep 1–40 flows/s, 2 runs each
Energy via tegrastats

Every flow is followed from send to verdict, queuing included. Sustained capacity: the highest rate at which every flow gets its verdict within 10 s of the load.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

└ III. RESULTS AND CONCLUSION

III. RESULTS AND CONCLUSION

III. RESULTS AND CONCLUSION

Result 1: splitting the pipeline sustains 9× the load

Mode	Flows/s	e2e p95 (s)	J / verdict
Single node	2.5	11.3	3.04
Single node + gate	1.9	7.8	3.04
B: Horizontal	3.3	8.2	3.20
C: Spark cluster	8.7	5.2	1.15
A: Split	22.4	7.1	0.60

Flows/s: sustained, every flow done within 10 s. e2e p95: send to verdict, queuing included.

- Split: 9× a single node, 5× less energy per verdict.
- The same gate on the same board gains nothing: the second board is what pays.
- Tail latency is seconds: one PySpark batch takes seconds.

Pipeline split is the recommended default: highest sustained load at the lowest energy per verdict.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 1: splitting the pipeline sustains 9× the load

Single node and horizontal run with the gate off, as in the submitted version. Energy is measured at each mode’s sustained rate: 2.0, 2.0, 3.9, 9.5 and 18.4 flows per second. Gate skip is about 89 percent in every gated mode. Median latency in split mode is about one second.

Result 1: splitting the pipeline sustains 9× the load

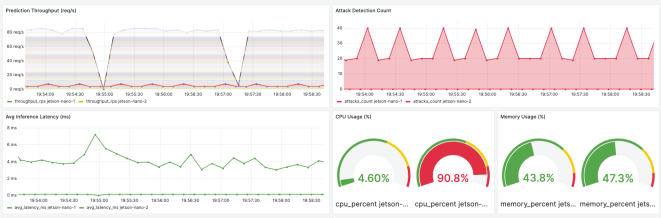
Mode	Flows/s	e2e p95 (s)	J / verdict
Single node	2.5	11.3	3.04
Single node + gate	1.9	7.8	3.04
B: Horizontal	3.3	8.2	3.20
C: Spark cluster	8.7	5.2	1.15
A: Split	22.4	7.1	0.60

Flows/s: sustained, every flow done within 10 s. e2e p95: send to verdict, queuing included.

Pipeline split is the recommended default: highest sustained load at the lowest energy per verdict.

- Split: 9× a single node, 5× less energy per verdict.
- The same gate on the same board gains nothing: the second board is what pays.
- Tail latency is seconds: one PySpark batch takes seconds.

Result 2: the load asymmetry is visible live



At peak load, split mode

(earlier run, previous model):

- gate node: **4.6% CPU**
- classifier node: **90.8% CPU**

The classifier dominates the compute budget — which is exactly why moving it off the ingest path pays.

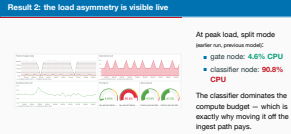
2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 2: the load asymmetry is visible live

Live Grafana at peak load. Under five versus ninety-one percent. The asymmetry is the whole argument for the split. Add: the gate is a tunable filter, not a fixed point — backup slide has the operating curve.



Result 3: what the gate lets through

Gate threshold (benign quantile)	Attacks forwarded	False alarms	Flows skipped
0.99	72.4%	0.99%	88.3%
0.995 (deployed)	68.1%	0.49%	89.4%
0.999	39.8%	0.09%	94.0%

Online: CICIDS2017 test split; gate on all 77 features.
Streamed live: 69.6% of 1,261 attack flows caught.

- A flow the gate skips is final: **about 30% of attack flows** never reach the classifier.
- On the classifier’s 30 features the gate would forward only 44.8%.
- The operator picks the point that fits the budget.

The capacity gain is paid for in recall: the gate is a **tunable trade-off**, not a free filter.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 3: what the gate lets through

Numbers from results/ml_08_anomaly_gate/gate_operating_points.csv; the deployed threshold is quantile 0.995. Without the gate, every attack flow in the replay was caught, so the loss is the gate’s.

Result 3: what the gate lets through

Gate threshold (benign quantile)	Attacks forwarded	False alarms	Flows skipped
0.99	72.4%	0.99%	88.3%
0.995 (deployed)	68.1%	0.49%	89.4%
0.999	39.8%	0.09%	94.0%

Online: CICIDS2017 test split; gate on all 77 features.
Streamed live: 69.6% of 1,261 attack flows caught.

- A flow the gate skips is final: **about 30% of attack flows** never reach the classifier.
- On the classifier’s 30 features the gate would forward only 44.8%.
- The operator picks the point that fits the budget.

Result 4: the price of one unified stack

We serve with Spark for consistency with distributed training. What does that cost? The *deployed forest*, exported, same board:

Engine	Flows/s	Batch p95 (ms)	mJ per flow
PySpark (deployed)	1.4	7515	1506
NumPy	54.8	194	11.3
ONNX Runtime	10,092	1.2	0.087

Identical predictions on every flow. mJ per flow: energy above the board's idle power.

ONNX Runtime serves the same forest $\approx 7,200\times$ faster. Target for the next iteration: **train on Spark, serve with ONNX/TensorRT**.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Result 4: the price of one unified stack

The JVM costs us more than three orders of magnitude, and that sets the roadmap. Batch size 10.

Result 4: the price of one unified stack

We serve with Spark for consistency with distributed training. What does that cost? The *deployed forest*, exported, same board:

Engine	Flows/s	Batch p95 (ms)	mJ per flow
PySpark (deployed)	1.4	7515	1506
NumPy	54.8	194	11.3
ONNX Runtime	10,092	1.2	0.087

Identical predictions on every flow. mJ per flow: energy above the board's idle power.

ONNX Runtime serves the same forest $\approx 7,200\times$ faster. Target for the next iteration: [train on Spark, serve with ONNX/TensorRT](#).

Conclusion

What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Gate and classifier on separate boards: **2.5 → 22.4 flows/s** sustained, e2e p95 $\approx$ 7 s.
- Energy per verdict: **3.0 J → 0.60 J**.
- ONNX serves the same forest $\approx$ 7,200 $\times$ faster.

Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- Gate skips $\approx$ 30% of attack flows.
- GPU unused (CPU-only pipeline); evasion untested.

Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

Thank you — questions?

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

III. RESULTS AND CONCLUSION

Conclusion

Recap: nine times the load, five times less energy per verdict, and the price, about thirty percent of attacks skipped by the gate.

Conclusion

What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Gate and classifier on separate boards: **2.5 → 22.4 flows/s** sustained, e2e p95 $\approx$ 7 s.
- Energy per verdict: **3.0 J → 0.60 J**.
- ONNX serves the same forest $\approx$ 7,200 $\times$ faster.

Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- Gate skips $\approx$ 30% of attack flows.
- GPU unused (CPU-only pipeline); evasion untested.

Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

Thank you — questions?

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **edge setting**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **anomaly gate**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP Top-30**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

[7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **closest system**

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage-aware protocol**

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

└─III. RESULTS AND CONCLUSION

└─Key references

Not presented — shown while taking questions, so sources are on screen if the audience asks. Full list is in the paper.

Key references	
[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP 2018.	CICIDS2017
[2] Shi, W. et al. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3(5), 2016.	edge setting
[3] Chandola, V., Banerjee, A. & Kumar, V. Anomaly detection: A survey. ACM Computing Surveys 41(3), 2009.	anomaly gate
[4] Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. NeurIPS 2017.	SHAP Top-30
[5] Zaharia, M. et al. Spark: Cluster computing with working sets. HotCloud 2010.	Spark
[6] Kreps, J., Narkhede, N. & Rao, J. Kafka: a Distributed Messaging System for Log Processing. NetDB 2011.	Kafka
[7] Mohan Kiran Kumar, K. et al. Distributed Intrusion Detection System using Kafka and Spark Streaming. ICVADV 2025.	closest system
[8] Arp, D. et al. Dos and Don'ts of Machine Learning in Computer Security. USENIX Security 2022.	leakage-aware protocol

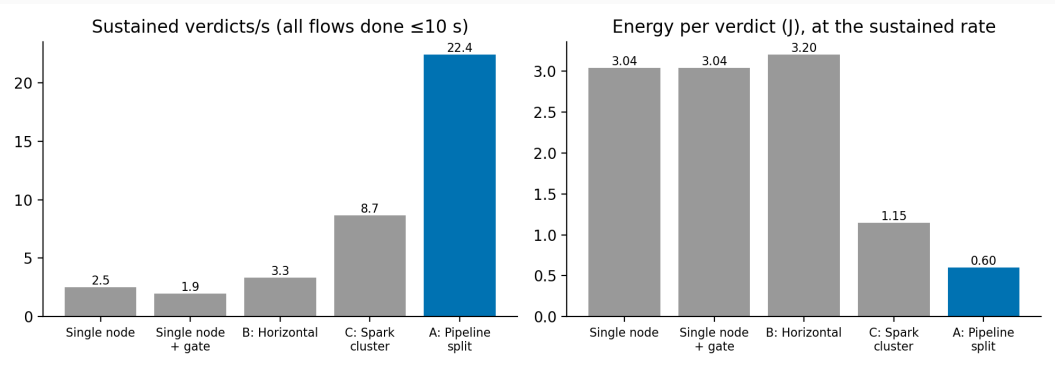
2026-09-30

A Distributed Real-Time Intrusion Detection System on a
Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup slides

Backup slides

Backup: sustained capacity and energy per mode

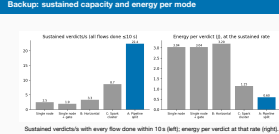


Sustained verdicts/s with every flow done within 10 s (left); energy per verdict at that rate (right).

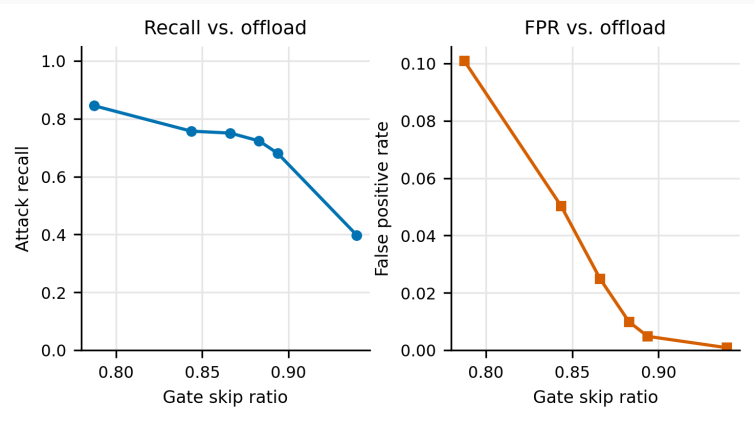
2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: sustained capacity and energy per mode



Backup: the gate is an operating curve, not a point

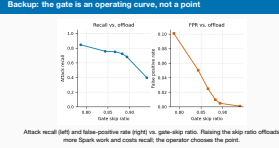


Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall; the operator chooses the point.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: the gate is an operating curve, not a point



Backup: which model, and why

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003 [†]	Explainable; de- ployed feature set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduc- tion

We deploy **SHAP Top-30 with Random Forest** (exported model: test F1 0.9959, 30 features): a native Spark ML estimator, so the PipelineModel loads on ARM64 without the native libraries Spark's XGBoost integration needs. The SHAP ranking is computed after destination_port is removed.

[†] Distributed saving under-reports size; true size ≈2.5 MB.

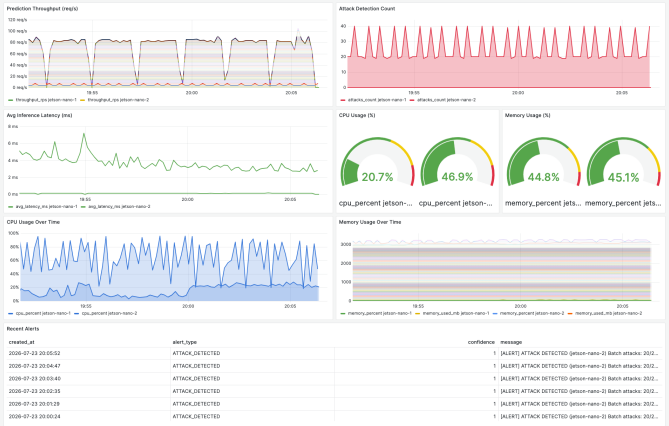
2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: which model, and why

Backup: which model, and why				
Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003 [†]	Explainable; de- ployed feature set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduc- tion
We deploy SHAP Top-30 with Random Forest (exported model: test F1 0.9959, 30 features): a native Spark ML estimator, so the PipelineModel loads on ARM64 without the native libraries Spark's XGBoost integration needs. The SHAP ranking is computed after destination_port is removed.				
[†] Distributed saving under-reports size; true size ≈2.5 MB.				

Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards,
PostgreSQL-backed alerts.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: full monitoring dashboard

Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards,
PostgreSQL-backed alerts.

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects every verdict.
- Each verdict stores the flow’s send time and replay row: latency is send to verdict, *queueing included*; skip ratio and attack recall come from the flows themselves.
- Rate sweep per mode; a rate is sustained when every flow sent in the 45 s window gets its verdict within 10 s after it. 2 runs per rate.
- Fresh Kafka consumer group per run, so no run inherits a backlog.
- Tiers batch 20 flows or flush after 1 s.
- Energy: tegrastats, 30 s idle baseline then load at the sustained rate; two-node modes sum both boards.

2026-09-30

A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects every verdict.
- Each verdict stores the flow’s send time and replay row: latency is send to verdict, *queueing included*; skip ratio and attack recall come from the flows themselves.
- Rate sweep per mode; a rate is sustained when every flow sent in the 45 s window gets its verdict within 10 s after it. 2 runs per rate.
- Fresh Kafka consumer group per run, so no run inherits a backlog.
- Tiers batch 20 flows or flush after 1 s.
- Energy: tegrastats, 30 s idle baseline then load at the sustained rate; two-node modes sum both boards.